NOI A.G. / S.p.A.
Roberto Cavaliere
r.cavaliere@noi.bz.it
T +39 0471 066 676

Bike Boxen STA (Loewenbytes)

v1.1, 29.07.2026

Preliminary notes

STA is implementing a network of bike parking systems in South Tyrol, to be placed mainly just outside train stations. Beside the supplier Bicincittà, integrated in the Open Data Hub since years, STA has now a new supplier, which is Loewenbytes (https://www.euroform-w.com/it/parcheggio-bici/bike-app) .

Loewenbytes has prepared an API for the integration of the real-time parking occupancy in the Open Data Hub, so that this information can be shown e.g. on suedtirolmobil applications.

Loewenbytes end-point description

The end-point is available at the following URL (testing environment): https://test.app.loewenbytes.com/api/v1

The API description is provided in form of a YAML file. The access to the API is managed via OAuth 2.0 client credentials workflow, we got client_id and client_secret for their testing environment.

The API implemented by Loewenbytes supports different use cases, including booking. For the Open Data Hub, only the method /resources is relevant. More specifically, the following services have to be considered:

- **/resources/locations**: this API call allows to retrieve the cities where the parking service is present (not only South Tyrol)
- **/resources/stations**: this API call allows to retrieve the list of stations present in the specified city and its overall real-time state.
- **/resources/station**: this API call allows to retrieve the list of parking slots associated to a station and its real-time state.

- **/resources/availability**: similar to /station, this API call allows to retrieve historical occupancy data .

Please note that the API implementation is very similar as for Bicincittà, so it could be a strategy to adapt the correspondent Data Collector for this integration.

Specification of the modalities of integration in the Open Data Hub

The Data Collector could directly call the method /resources/stations to get the details of all associated stations, without considering the call /locations. Bike parking stations are probably represented separately in the API and should not be modeled as being part of the same parking facility (e.g., a station as veloHub and a station as bike boxen group).

Please note that the web-service provides names in different languages; all this information should be properly stored (one language for the "default" station name in the Open Data Hub; all other languages available in the metadata data structure).

METADATA

The following proposed mapping takes as reference the fields provided by the method /resources/stations. All fields marked as "METADATA" indicate the necessity to have a linked record in the metadata table in which these values have to be stored.

Web-service fields	Open Data Hub parameters
locationID	METADATA
locationName	METADATA
stationID	stationcode
name (languageID = it)	name
name (languageID = de)	METADATA
name (languageID = lld)	METADATA
name (languageID = en)	METADATA
address	METADATA
latitude, longitude	pointprojection
type	METADATA. Store directly the associated mapping (4 = veloHub, 5 = bixeBoxGroup) Note that this metadata is provided through the method /resources/locations
totalPlaces	METADATA
places	METADATA

Table 1: Mapping between /stations and Open Data Hub metadata tables.

The following specifications have to be also considered:

- the Open Data Hub field **origin** is to set as **loewenbytes**.

- the Open Data Hub field **stationtype** is to set as **BikeParking**

REAL-TIME DATA

The associated real-time state for the two types of stations is provided through certain fields in the above mentioned methods. Where possible, existing types already available in the Open Data Hub should be reused. The following measurements have to be stored, with reference the type of data and the associated station.

Web-service fields (/stations)	Measurement type
state	Existing type ‚**usage state**'. Reference mapping to be stored (1=in service, 2= out of service) Table: **measurementstring**
countFreePlacesAvailable _MuscularBikes	New type ‚**Free parking spots (regular bikes)**'. Table: **measurement**
countFreePlacesAvailable _AssistedBikes	New type ‚**Free parking spots (electric bikes)**'. Table: **measurement**
countFreePlacesAvailable	Existing type ‚**free**'. Table: **measurement**

Table 2: Measurements to be stored for a bike parking station.